

Matrix Strategies with github actions

A **Matrix Strategy** in GitHub Actions lets you execute the same job multiple times with different combinations of variables (operating systems, Node.js versions, browsers, environments, etc.). It's one of the most powerful features for CI/CD because it enables parallel testing across multiple configurations.

Basic Syntax

</> YAML 

```
jobs:
  test:
    runs-on: ${{ matrix.os }}

    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]

    steps:
      - uses: actions/checkout@v4

      - run: echo "Running on ${{ matrix.os }}"
```

Execution

GitHub automatically creates **3 jobs**:

```
test (ubuntu-latest)
test (windows-latest)
test (macos-latest)
```



All run in parallel.

Example 1: Multiple Node.js Versions

</> YAML



```
name: Node Matrix

on:
  push:
    branches:
      - main

jobs:
  test:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        node-version: [18, 20, 22]

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: ${ matrix.node-version }

      - run: npm ci
      - run: npm test
```

Generated Jobs

Node 18
Node 20
Node 22



Example 2: Multiple Operating Systems + Node Versions

</> YAML



```
strategy:
  matrix:
    os:
      - ubuntu-latest
      - windows-latest
      - macos-latest

    node-version:
```

- 18
- 20

</> YAML



```
runs-on: ${{ matrix.os }}
```

GitHub creates every possible combination.

OS	Node
Ubuntu	18
Ubuntu	20
Windows	18
Windows	20
macOS	18
macOS	20

Total Jobs:

3 × 2 = 6 Jobs



Complete Workflow

</> YAML



```
name: Matrix Demo

on:
  push:

jobs:
  build:
    runs-on: ${{ matrix.os }}

    strategy:
      matrix:
        os:
          - ubuntu-latest
```

```
      - windows-latest
      - macos-latest

    node-version:
      - 18
      - 20

  steps:
    - uses: actions/checkout@v4

    - uses: actions/setup-node@v4
      with:
        node-version: ${{ matrix.node-version }}

    - run: node -v
    - run: npm ci
    - run: npm test
```

Example 3: Angular Application

</> YAML



```
strategy:
  matrix:
    node-version: [20, 22]
```

</> YAML



```
steps:
  - uses: actions/checkout@v4

  - uses: actions/setup-node@v4
    with:
      node-version: ${{ matrix.node-version }}

  - run: npm ci

  - run: npm run lint

  - run: npm run test -- --watch=false

  - run: npm run build
```

This ensures your Angular application works with multiple Node.js versions.

Example 4: Browser Testing

</> YAML



```
strategy:
  matrix:
    browser:
      - chrome
      - firefox
      - edge
```

</> YAML



```
- run: npm run test:${{ matrix.browser }}
```

Generated jobs:

Chrome
Firefox
Edge



Example 5: Deployment Environments

</> YAML



```
strategy:
  matrix:
    environment:
      - dev
      - qa
      - stage
```

</> YAML



```
steps:
  - run: echo "Deploying to ${{ matrix.environment }}"
```

Jobs:

Deploy Dev
Deploy QA
Deploy Stage



Example 6: Include Extra Values

Sometimes one configuration needs additional variables.

 **YAML** 

```
strategy:
  matrix:
    os:
      - ubuntu-latest
      - windows-latest

    include:
      - os: ubuntu-latest
        database: mysql

      - os: windows-latest
        database: sqlserver
```

Use it like this:

 **YAML** 

```
- run: echo ${ matrix.database }
```

Result:

OS	Database
Ubuntu	MySQL
Windows	SQL Server

Example 7: Exclude Combinations

Suppose macOS doesn't support Node 18.

 **YAML** 

```
strategy:
  matrix:
    os:
      - ubuntu-latest
      - windows-latest
```

```
- macos-latest

node-version:
  - 18
  - 20

exclude:
  - os: macos-latest
    node-version: 18
```

Generated jobs:

```
Ubuntu-18
Ubuntu-20
Windows-18
Windows-20
macOS-20
```



Only 5 jobs instead of 6.

Example 8: Include New Combination

</> YAML



```
strategy:
  matrix:
    os:
      - ubuntu-latest

    node-version:
      - 20

  include:
    - os: windows-latest
      node-version: 22
```

Jobs:

```
Ubuntu + Node20
Windows + Node22
```



Example 9: Fail Fast

By default, if one matrix job fails, GitHub cancels the remaining matrix jobs.

YAML



```
strategy:
  fail-fast: true
```

Disable this behavior:

YAML



```
strategy:
  fail-fast: false
```

Now every job runs to completion, even if one fails.

Example 10: Max Parallel Jobs

Limit how many matrix jobs run simultaneously.

YAML



```
strategy:
  max-parallel: 2

matrix:
  node-version: [18, 20, 22, 24]
```

Execution:

Job1 (18) 
Job2 (20) 



After completion:

Job3 (22)
Job4 (24)

Only **2 jobs** run at the same time.

Accessing Matrix Variables

</> YAML



```
${{ matrix.os }}

${{ matrix.node-version }}

${{ matrix.browser }}

${{ matrix.environment }}
```

Example:

</> YAML



```
- run: echo "OS = ${{ matrix.os }}"

- run: echo "Node = ${{ matrix.node-version }}"
```

Real Enterprise Example

</> YAML



```
name: Enterprise CI

on:
  pull_request:

jobs:
  test:
    runs-on: ${{ matrix.os }}

    strategy:
      fail-fast: false
      max-parallel: 3

    matrix:
      os:
        - ubuntu-latest
        - windows-latest

      node-version:
        - 20
        - 22

    include:
      - os: ubuntu-latest
```

```

      node-version: 20
      coverage: true

    exclude:
      - os: windows-latest
        node-version: 20

  steps:
    - uses: actions/checkout@v4

    - uses: actions/setup-node@v4
      with:
        node-version: ${{ matrix.node-version }}

    - run: npm ci
    - run: npm test

    - name: Coverage
      if: matrix.coverage == true
      run: npm run coverage

```

Generated jobs:

OS	Node	Coverage
Ubuntu	20	✓
Ubuntu	22	✗
Windows	22	✗

Common Use Cases

Use Case	Matrix Variable
Test multiple Node.js versions	node-version
Cross-platform testing	os
Browser compatibility	browser
Database testing	database
Multiple Java versions	java-version

Use Case	Matrix Variable
Python versions	<code>python-version</code>
Deployment targets	<code>environment</code>
Regions (AWS/Azure/GCP)	<code>region</code>

Key Benefits

- **Parallel execution** reduces total CI time.
- **Cross-platform validation** ensures compatibility across operating systems.
- **Version compatibility testing** (Node.js, Java, Python, etc.) is easy to configure.
- **Flexible customization** using `include` and `exclude`.
- **Resource control** with `max-parallel`.
- **Reliable reporting** with `fail-fast: false` when you want results from every configuration.

